

Comparing LoRA, Adapters, and BatchNorm Tuning

for Food-101 Classification

INAARA RAJWANI • PETER NGUYEN • AUSTIN COULTER

DATA 586 – Advanced Machine Learning

Problem & Motivation



The Challenge

Fine-tuning modern image classification models presents significant hurdles:

- High compute costs for full training
- Prohibitively expensive resources
- *How can we adapt models efficiently?*



Research Goal

Optimize Efficiency without Sacrifice

Compare parameter-efficient fine-tuning (PEFT) methods specifically for:

- **Accuracy**
- **Robustness**

Dataset & Methodology



Dataset: Food-101

- 101 food categories
- Real-world images
- Fine-grained classification



Train/Test Configuration

Train: Food101

- Test: Clean, Masked, Blurred
- Downsampled, Noise, rotation



Technical Approach: Backbones & Methods



Backbones

ResNet18:

Classic residual network (11M params).

EfficientNetV2-S:

Modern architecture for efficiency.



Fine-Tuning Strategies

Linear Probing: Baseline approach.

LoRA: Rank-decomposition matrices.

Adapters: Bottleneck layers.

BatchNorm: Scale/shift updates.

Custom: ResNet18 + CBAM.



Evaluation Metrics

- Accuracy
- Robustness
- Efficiency

We compare these metrics across all models and tuning methods.



The diagram illustrates a residual block architecture. It starts with an 'Input' which is processed by a 'Convolution' layer. The output of the 'Convolution' layer is then split: one path goes through a 'ReLU' layer, and the other path is a 'skip connection' that bypasses the 'ReLU' layer. The outputs of the 'ReLU' layer and the 'skip connection' are then combined in an 'Addition' layer. The final output of the 'Addition' layer is the 'Output'.



The diagram illustrates the architecture of the proposed Fused-MBConv module. It shows a sequence of operations: an initial MBConv layer, followed by a 3x3 Conv layer, then another MBConv layer. This is followed by a 1x1 Conv layer, then a 3x3 Conv layer, and finally another MBConv layer. The output of this sequence is fed into the Fused-MBConv module, which consists of an MBConv layer, a Fused-MBConv layer, and another Fused-MBConv layer. The output of the Fused-MBConv module is then passed to a Pooling layer.

Both models utilize ImageNet pretrained weights to leverage learned feature hierarchies.

Fine-Tuning Strategies



Methods Overview

BatchNorm Tuning:

Scale/shift updates in norm layers.

Adapters:

Small bottleneck layers in backbone.

LoRA:

Rank-decomposition matrix injection.



LoRA Deep Dive

LoRA (Our Approach)

Core Idea & Implementation

- Freeze backbone, inject low-rank convolution updates
- Applied to extractor; excluded depthwise layers
- Integration via `loralib`

Insights

Reduces trainable parameters while adapting pretrained features

Observation: Did not outperform BatchNorm tuning in our project.

Reference implementation in `helpers.py` (`build_resnet18_lora` / `build_efficientnet_v2_s_lora`)

Custom Model



Methods Overview

CBAM Attention Layer

Strategic integration for enhanced feature focus.

Task Specific Adapters

Small bottleneck layers in backbone.

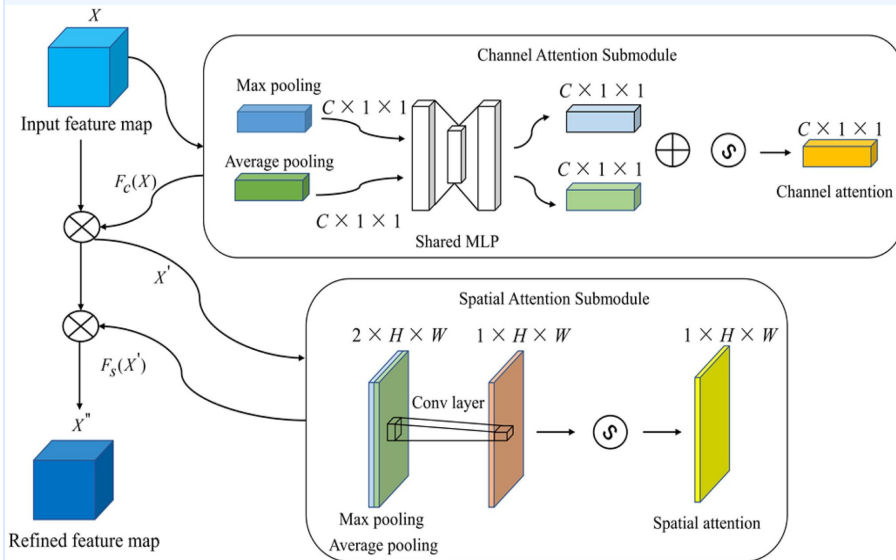
Batch Norm Tuning

Rank-decomposition matrix injection.

Results

5% increase in accuracy compared to TSA or BN tuning alone.

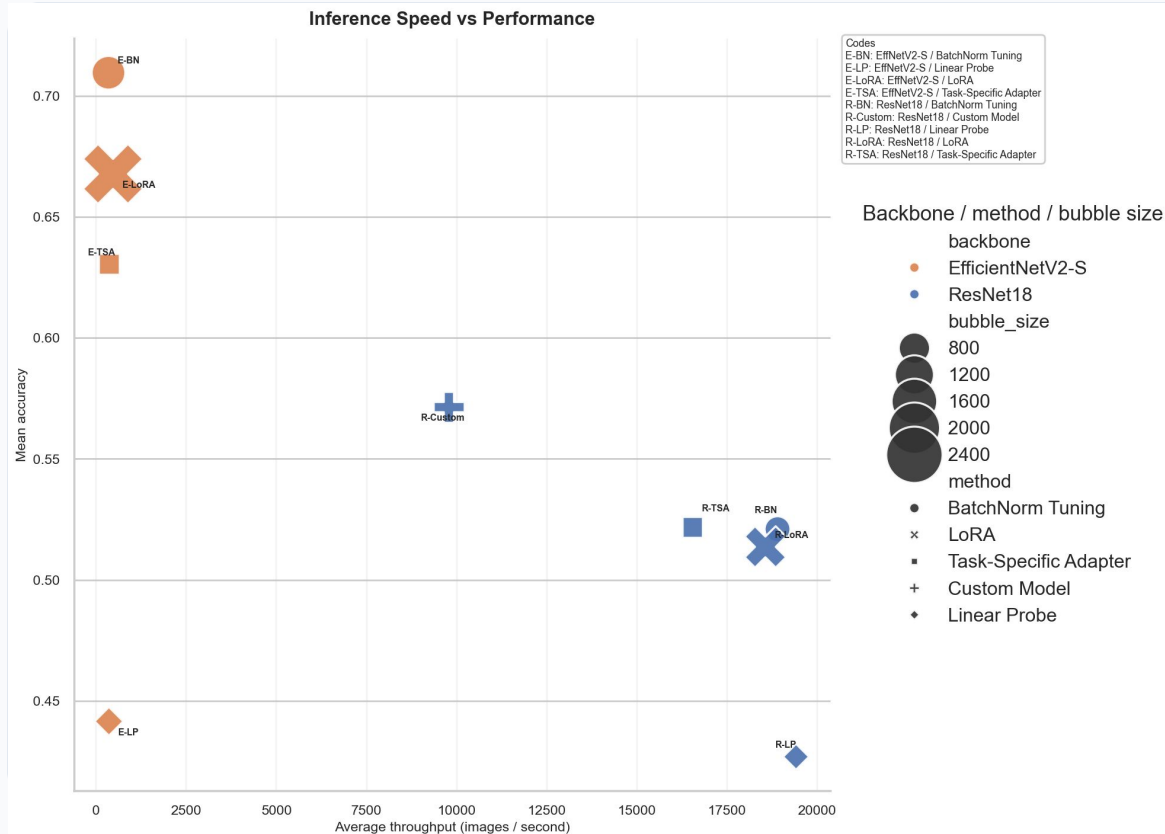
CBAM Layer Architecture



Performance Results - Accuracy

Backbone	Method	Mean Acc.	Clean	Masked	Downsampled
ResNet18	Linear probe	42.69%	57.58%	50.86%	17.17%
ResNet18	BatchNorm tuning	52.10%	67.87%	63.58%	22.95%
ResNet18	Task-adapter	52.18%	67.37%	63.09%	22.86%
ResNet18	LoRA	51.36%	67.38%	63.71%	20.13%
ResNet18	Custom model	57.15%	69.56%	68.31%	25.27%
EffNetV2-S	Linear probe	44.15%	61.77%	60.91%	10.61%
EffNetV2-S	BatchNorm tuning	70.95%	86.47%	85.79%	29.72%
EffNetV2-S	Task-adapter	63.04%	80.65%	79.86%	25.64%
EffNetV2-S	LoRA	66.77%	84.94%	83.95%	23.98%

Performance vs Inference Cost Trade-off



Key Insights

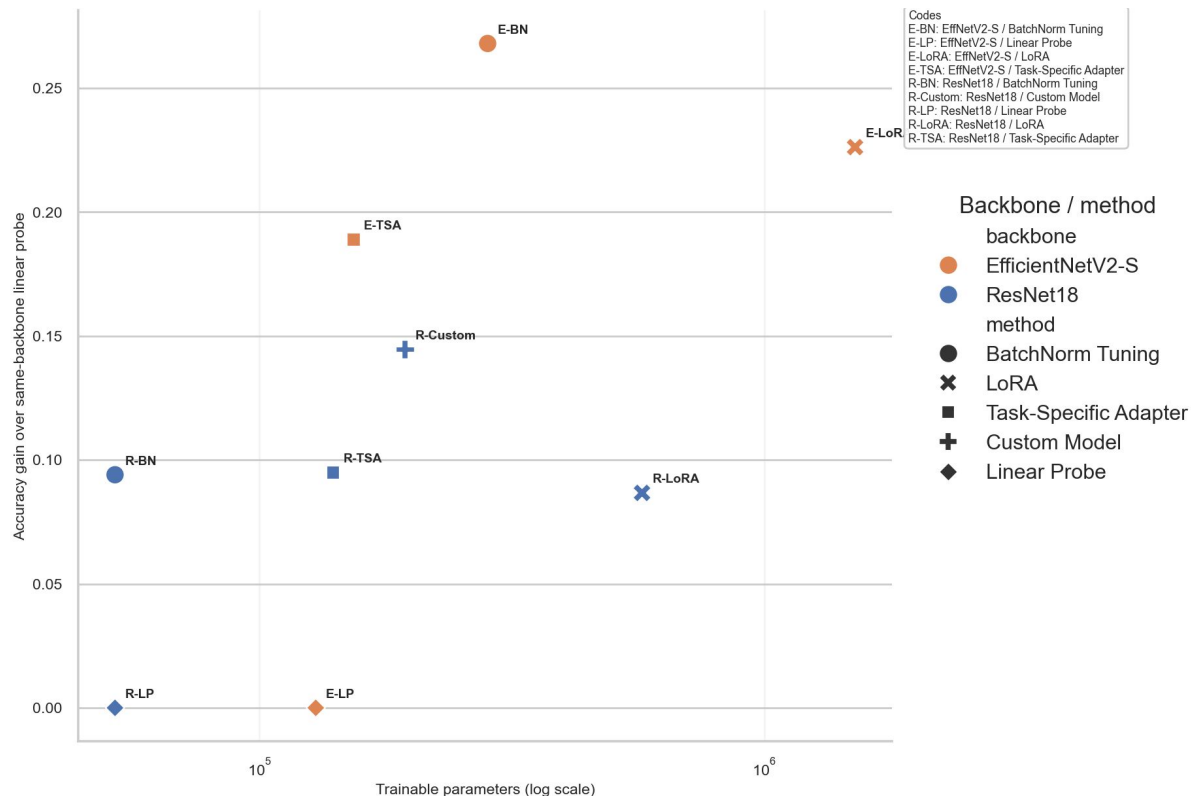
TOP PERFORMER

EfficientNetV2-S + BatchNorm tuning achieves highest accuracy (>70%).

EFFICIENCY

ResNet is up to 30x faster, though EfficientNet is more accurate.

Accuracy vs. Trainable Parameters



Key Insights



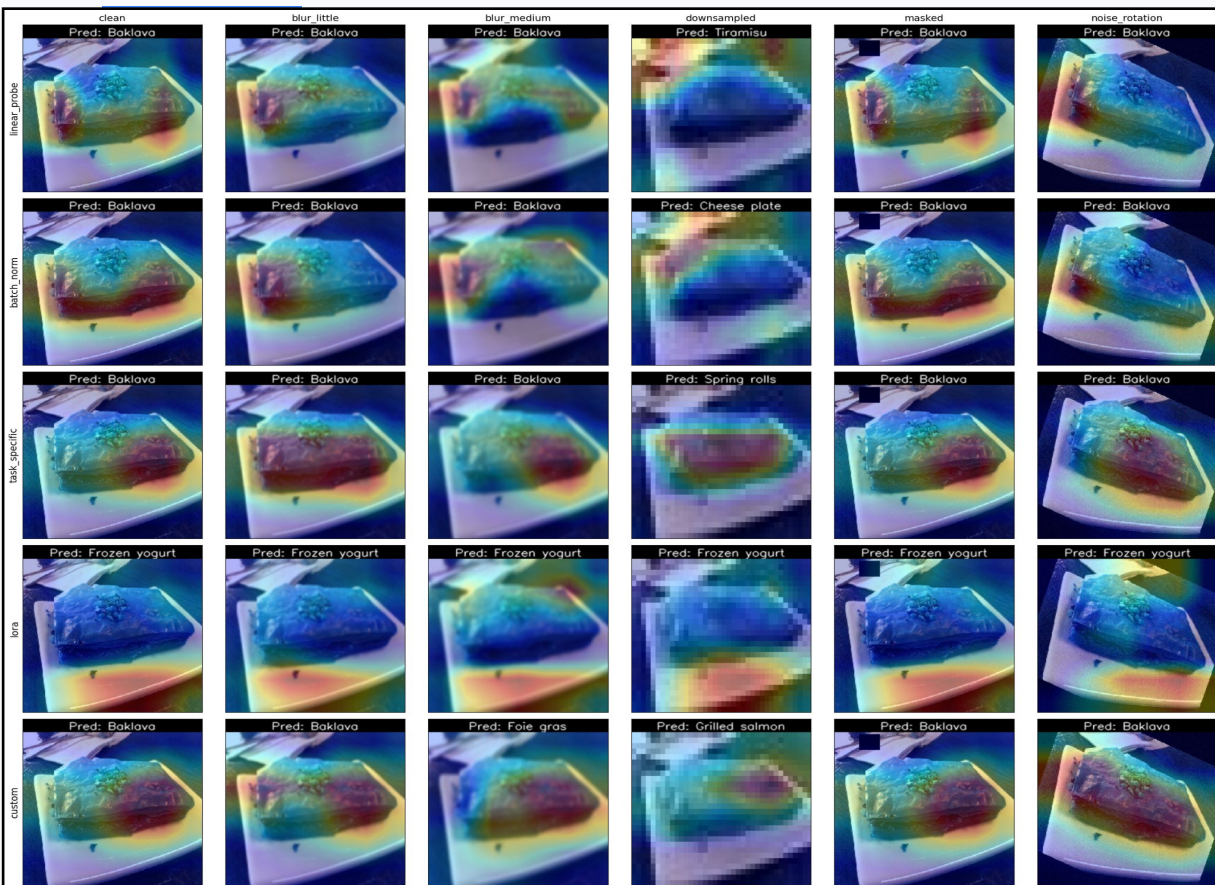
General Trend: More trainable parameters typically lead to higher accuracy.

EXCEPTION: LoRA

Parameter-efficient but didn't outperform BatchNorm tuning.

Parameter placement matters more than count.

Grad-CAM: RESNET-18



Warmer regions indicate higher importance for prediction.

Linear Probing:

Inconsistent attention, but generally highlights the food.

BatchNorm Tuning:

Improved focus but unstable under distortion.

Adapters:

High stability; focuses effectively on the food region.

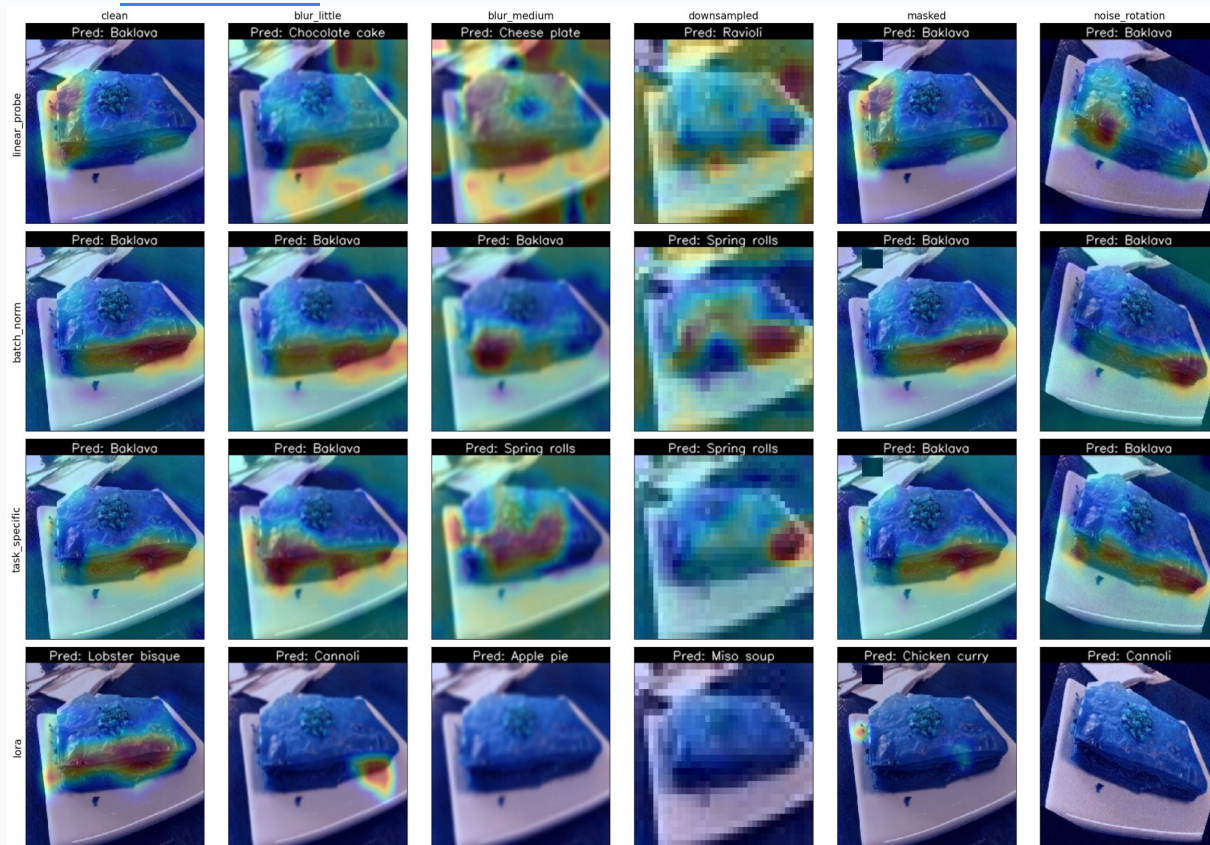
LoRA:

Consistently highlights the background region.

Custom Model:

Top performer; consistent focus and robustness.

Grad-CAM: EFFICIENTNET V2



Linear Probe:

Scattered attention; often focuses on background elements.

BatchNorm & Adapters:

Improved focus, but attention remains inconsistent under corruptions.

LoRA:

Produces worse activations and are almost non existent under corruptions.

Conclusion

Performance Improvement

Parameter-efficient fine-tuning improves performance over baseline.

Top Performer

EfficientNetV2-S with BatchNorm achieved the highest accuracy.

Best Robustness

Custom ResNet18 model showed the best robustness.

Operational Trade-offs

Clear trade-offs between accuracy, speed, and model complexity.

Visual Confirmation

Grad-CAM confirms improved attention leads to better predictions.